

Data Warehouse scolaire : Airflow + Spark vers MongoDB

ÉTI sur le Data Warehouse : dimensions, faits et contrôle qualité

Projet – Ingénierie de données

Septembre 2026

Contexte et problématique

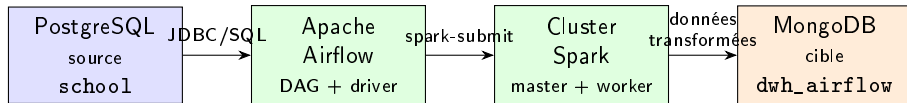
- Une base **PostgreSQL** gère la scolarité : étudiants, classes, matières, professeurs, notes (système *d'exploitation*).
- Ce système est mal adapté à l'**analyse** : schéma normalisé, requêtes lourdes, pas d'indicateurs prêts à l'emploi.
- Le projet doit fournir un **Data Warehouse** scolaire exploitable par une direction (moyennes, taux de réussite).

Problématique

Comment alimenter automatiquement un entrepôt de données fiable, à partir d'une base de gestion, en garantissant la qualité des données ?

- ❶ **Modéliser** les données selon un schéma *dimensionnel en étoile* (6 dimensions + table de faits + indicateurs).
- ❷ **Automatiser** l'alimentation par un *pipeline ETL* planifié (DAG Airflow → *@daily*).
- ❸ **Sécuriser** le chargement par un *contrôle de qualité* avec court-circuit automatique.
- ❹ **Industrialiser** : calcul distribué (Spark), environnements reproductibles (Docker Compose).

Architecture cible



- **Source** : PostgreSQL school (23 940 notes, 1 000 étudiants).
- **Orchestration** : DAG dwh_scolaire_postgres_to_mongo.
- **Calcul** : Spark standalone (2 cœurs, 1 Go – volontairement modeste).
- **Cible** : MongoDB dwh_airflow (modèle document).

Stack technique et rôles

Outil	Rôle dans le projet
PostgreSQL 13	base <i>source</i> (gestion scolaire) + base de métadonnées d'Airflow
Apache Airflow 2.10	orchestration, planification, dépendances, retries, court-circuit
Apache Spark 3.5 + PySpark	calcul distribué : contrôle qualité, faits, agrégats
MongoDB 7	entrepôt cible (dimensions, faits, indicateurs, QC)
Docker Compose	8 services conteneurisés, environnement reproductible
pymongo	écriture MongoDB depuis le driver (lots de 1000 documents)
JDBC (driver 42.7.3)	lecture partitionnée de la source dans Spark
Interfaces	Airflow UI 8080, Spark UI 8081, pgAdmin 5050, mongo-express 8082

Modèle dimensionnel (étoile)

Objet	Cardinalité
dim_etudiant	1 000
dim_enseignant	60
dim_matiere	30
dim_classe	24
dim_temps	289
dim_type_evaluation	2
fait_notes	23 940
agg_moyenne_par_matiere	30
agg_moyenne_par_etudiant	1 000
qc_report	1

Document fait_notes

```
{
  "id_fait" : 17370,
  "note" : 12.73,
  "credit" : 3,
  "id_DimEtudiant_FK" : "ETU2025-00729",
  "id_DimMatiere_FK" : 7,
  "id_DimClasse_FK" : 1,
  "id_DimTemps_Fk" : 2,
  "id_DimTypeEvaluation_FK" : 2
}
```

Dimensions = axes d'analyse ; faits = mesures. *Note* : pas de dimension enseignant dans le fait (source sans information de professeur par note).

Pipeline du DAG

- ➊ **Dimensions (SQL)** : 6 tâches parallèles chargent les référentiels (drop + rechargement total, écriture `replace_many`).
- ➋ **Contrôle qualité (PySpark)** : `run_pyspark_qc` – comptage, valeurs manquantes, taux < seuil 20 % (3,05 % mesurés) → rapport OK.
- ➌ **Garde-fou** : `check_qualite` (*ShortCircuitOperator*) – si anomalie, toutes les tâches aval passent en skipped.
- ➍ **Chargement (PySpark)** : `run_pyspark_load` – lecture JDBC partitionnée (sur `id_matiere`), construction des clés de substitution, `fait_notes` + 2 agrégats, écriture par lots de 1000 (driver).

Caractéristiques

Planification @daily, catchup=False, 1 essai après 5 minutes ; mode standalone, exécuteurs sur le worker, *driver* dans le conteneur Airflow.

Contrôle qualité et court-circuit

Mesures QC (mode qc) :

- notes totales : **23 940**
- valeurs manquantes : **730**
- taux : **3,05 %** < seuil **20 %**
- statut : **OK**

Mécanisme : ShortCircuitOperator relit qc_report.

- rapport absent → échec de la tâche ;
- statut ANOMALIE → suite du DAG skipped.

La barrière a été testée en abaissant le seuil : le charger de fait n'a pas été relancé.

Sans garde-fou

Des notes manquantes à plus de 20 % alimenteraient un entrepôt faussé.

Avec garde-fou

Seules des données conformes atteignent fait_notes et les indicateurs.

Exécutions validées :

- run manuel + run planifié : pipelines **success** ;
- 10 collections alimentées (intégralité du tableau vu plus haut) ;
- report qc_report final : **OK**.

Reproductibilité (3 commandes) :

- `docker compose up -d`
- `docker compose up airflow-init -wait`
- `init_connections.sh` (connexions source, cible, Spark)

Durées observées (local)

- contrôle qualité : $\approx 1 \text{ min } 30$
- chargement des faits : $\approx 4 \text{ min}$

Accès

Airflow UI 8080 · Spark UI 8081
pgAdmin 5050 · mongo-express 8082

Points techniques clés

- **JDBC** : le fichier jars ne suffit pas ; ajout de `driver_class_path` car la lecture s'exécute dans le *driver*.
- **Connexion Spark** : le provider reconstruit l'URL en `host:port` → le host doit contenir le schéma : `spark://spark-master`, port 7077.
- **Image Apache Spark** : scripts dans `/opt/spark/bin` (et non `sbin`) ; lancement master/worker via `spark-class`.
- **Healthcheck master** : `/bin/sh = dash` → `test bash /dev/tcp`, sur le *nom d'hôte*, pas 127.0.0.1.
- **Écriture cible** : pymongo par lots de 1 000, en rechargement total (consistance garantie).

Limites assumées

- rechargement *total*, non incrémental ;
- écriture MongoDB côté *driver* (exécuteurs sans pymongo) ;
- identifiants visibles dans la commande spark-submit ;
- volume de test modeste (choix pédagogique).

Perspectives

- chargement *incrémental* (horodatage de modification) ;
- *connecteur Spark-MongoDB* officiel ;
- *gestionnaire de secrets* pour les connexions ;
- ajout de workers pour monter en charge.

Conclusion

- Une chaîne ETL complète : **PostgreSQL** → **Airflow** → **Spark** → **MongoDB**.
- Le **contrôle qualité** intégré bloque le chargement en cas d'anomalie (mécanisme de court-circuit).
- Un environnement **conteneurisé** et reproductible, des UIs de supervision intégrées.
- Un plan d'évolution clair vers une exploitation de production (incrémental, secrets, connecteur officiel).

Merci pour votre attention – questions ?